AIMLCZG546 – Software Engineering for Machine Learning

Assignment II

Group Details

Field	Value
Group No	146
Course	AIMLCZG546 – SEML

Group Contribution Declaration

Member Name	Student ID	Contribution (%)
Hakeem Sharath	2025AA05665	100%
Harisha	2025AA05669	100%
Azhar Ansari	2025AB05055	100%
Jadhav Vedant Bhagwan Swati	2025AA05002	100%

ML Application: Iris Flower Species Classification

Problem Statement: Classify Iris flowers into three species (Setosa, Versicolor, Virginica) based on four morphological measurements using a Random Forest classifier.

Dataset: Fisher's Iris dataset (150 samples, 4 features, 3 classes) – available via `sklearn.datasets`.

Project Structure:

```
Assignment-2/
├── src/
│   ├── data_ingestion.py      # Data loading, validation,
quality metrics
│   ├── feature_engineering.py # Scaling, sklearn Pipeline
construction
│   ├── model_training.py      # Training, evaluation,
persistence
│   ├── inference.py           # IrisClassifier wrapper class
│   └── api.py                 # FastAPI REST API
├── tests/
│   ├── test_unit.py           # Unit tests (per-function)
│   ├── test_integration.py    # End-to-end pipeline tests
```

```
│   ├── test_data_validation.py# Schema & quality tests
│   └── test_ml_components.py  # Training & inference ML tests
├── research_code/
│   └── prototype.py           # Research/exploratory code
(before refactoring)
└── 146.ipynb                  # This submission notebook
```

Setup: Install Dependencies & Configure Logging

In [1]:
```
# Install required packages (already installed in the venv)
import subprocess, sys
pkgs = ["scikit-learn", "pandas", "numpy", "fastapi", "uvicorn",
        "httpx", "pytest", "black", "flake8", "isort", "pydantic", "joblib"]
subprocess.run([sys.executable, "-m", "pip", "install", "-q"] + pkgs)
print("All packages available.")
```

```
All packages available.
```

In [2]:
```
import logging
import sys
import warnings
warnings.filterwarnings("ignore")

# Configure root logger so all src.* modules emit to stdout for demonstration
logging.basicConfig(
    stream=sys.stdout,
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger("notebook")
logger.info("Logging configured.")
```

```
17:58:00 | INFO     | notebook | Logging configured.
```

Objective 1 – Implementation and Code Sharing

1.1 OOP / Functional Design of the ML Application

The production codebase is organised into four **single-responsibility modules**:

Module	Responsibility
`src/data_ingestion.py`	Load dataset, validate schema & values, train/test split, data-quality metrics
`src/feature_engineering.py`	StandardScaler fitting, sklearn `Pipeline` construction, model persistence helpers
`src/model_training.py`	Model instantiation (`build_model`), training (`train_model`), evaluation (`evaluate_model`)
`src/inference.py`	`IrisClassifier` class – thread-safe wrapper for single and batch predictions

Key OOP principles applied:

- **Encapsulation**: `IrisClassifier` hides the pipeline and exposes only `load()`, `predict()`, `predict_batch()`.
- **Separation of Concerns**: Data ingestion, feature engineering, training, and inference are independent modules.
- **Single Responsibility**: Each function/class has one well-defined purpose.

In [3]:
```
# Show the IrisClassifier class definition (key OOP component)
import inspect
from src.inference import IrisClassifier
print(inspect.getsource(IrisClassifier))
```

```
class IrisClassifier:
    """Wraps a fitted sklearn Pipeline for thread-safe, logged inference."""

    def __init__(self, model_path: str = DEFAULT_MODEL_PATH) -> None:
        self.model_path = model_path
        self._pipeline: Pipeline | None = None

    def load(self) -> None:
        """Load the pipeline from disk."""
        logger.info("Loading model from '%s'.", self.model_path)
        self._pipeline = load_pipeline(self.model_path)
        logger.info("Model loaded. Classes: %s.", list(self._pipeline.classes_))

    @property
    def pipeline(self) -> Pipeline:
        if self._pipeline is None:
            raise RuntimeError("Model is not loaded. Call load() first.")
        return self._pipeline

    def predict(self, features: Dict[str, float]) -> Dict[str, Any]:
        """
        Predict the species for a single sample.

        Args:
            features: dict with keys matching FEATURE_NAMES.

        Returns:
            dict with 'predicted_class' and 'probabilities'.
        """
        logger.info("Predicting single sample: %s", features)
        try:
            df = _dict_to_dataframe(features)
            pred_class = self.pipeline.predict(df)[0]
            proba = self.pipeline.predict_proba(df)[0]
            result = {
                "predicted_class": pred_class,
                "probabilities": {
                    cls: round(float(p), 4)
                    for cls, p in zip(self.pipeline.classes_, proba)
                },
            }
            logger.info(
                "Prediction: %s (confidence %.2f%%)", pred_class, max(proba) * 10
0
            )
            return result
        except Exception as exc:
            logger.error("Inference failed: %s", exc)
            raise

    def predict_batch(self, samples: List[Dict[str, float]]) -> List[Dict[str, An
y]]:
        """
        Predict species for a list of samples.

        Args:
            samples: list of feature dicts.

        Returns:
            list of prediction dicts.
```

```
        """
        logger.info("Predicting batch of %d samples.", len(samples))
        if not samples:
            logger.warning("Empty batch received.")
            return []
        try:
            df = pd.DataFrame(samples)[FEATURE_NAMES]
            pred_classes = self.pipeline.predict(df)
            probas = self.pipeline.predict_proba(df)
            results = []
            for pred, proba in zip(pred_classes, probas):
                results.append(
                    {
                        "predicted_class": pred,
                        "probabilities": {
                            cls: round(float(p), 4)
                            for cls, p in zip(self.pipeline.classes_, proba)
                        },
                    }
                )
            logger.info("Batch prediction complete: %d results.", len(results))
            return results
        except Exception as exc:
            logger.error("Batch inference failed: %s", exc)
            raise
```

In [4]:
```
# Show the sklearn Pipeline factory (functional design)
from src.feature_engineering import create_feature_pipeline
print(inspect.getsource(create_feature_pipeline))
```

```
def create_feature_pipeline(model) -> Pipeline:
    """
    Wrap a classifier with a StandardScaler into an sklearn Pipeline.
    The pipeline ensures identical preprocessing during training and inference.
    """
    logger.info(
        "Creating sklearn Pipeline: [StandardScaler -> %s].", type(model).__name_
_
    )
    pipeline = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("classifier", model),
        ]
    )
    logger.info("Pipeline created successfully.")
    return pipeline
```

1.2 Research Code vs Production Code

The `research_code/prototype.py` file represents the **exploratory / notebook-style** code written during the research phase. It works, but has several weaknesses compared to the production code in `src/`.

Concern	Research Code (`prototype.py`)	Production Code (`src/`)
Structure	Flat script, procedural	Modular OOP with separate files
Error handling	None	`try/except` with meaningful messages
Logging	`print()` statements	`logging` module with levels
Preprocessing	Manual scaler, not in pipeline	`StandardScaler` inside `sklearn.Pipeline`
Schema validation	None	`validate_data()` checks columns, types, ranges, labels
Reusability	Script must be re-run manually	Functions callable from API and tests
Data leakage risk	High (scaler applied before split)	None (scaler fitted only on train set inside Pipeline)
Reproducibility	Partial (no stratify in split)	Full (stratified split, fixed `random_state`)

Research Code (prototype.py) – key excerpt:

```
# No validation, no logging – research style
iris = load_iris()
X = iris.data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)   # Leakage risk: scaler
outside pipeline
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
```

Production Code (src/model_training.py) – same concern handled properly:

```
def train_model(X_train, y_train, ...):
    logger.info("Starting model training with %d samples.",
len(X_train))
    clf = build_model(n_estimators=n_estimators,
random_state=random_state)
    pipeline = create_feature_pipeline(clf)   # scaler INSIDE pipeline
    pipeline.fit(X_train, y_train)
    save_pipeline(pipeline, model_path)
    return pipeline
```

In [5]:
```
# Print the full research prototype for reference
with open("research_code/prototype.py") as f:
    print(f.read())
```

```
"""
RESEARCH / PROTOTYPE CODE - Before Refactoring
This is the exploratory, notebook-style code written during the research phase.
It works, but lacks proper structure, error handling, logging, and separation of
concerns.
Contrast this with the production code in src/.
"""

# %%
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib

# %%  Load data - no validation, no logging
iris = load_iris()
X = iris.data
y = iris.target
print("Data shape:", X.shape)
print("Classes:", iris.target_names)

# %% Quick EDA
df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y
print(df.describe())
print(df['target'].value_counts())

# %% Split - hardcoded params, no stratify
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s
tate=42)

# %% Scale - manual, not in pipeline
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# %% Train model - magic numbers, no logging
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# %% Evaluate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# %% Save - no error handling
joblib.dump({'clf': clf, 'scaler': scaler}, 'research_code/model.pkl')
print("Saved!")

# %% Predict new sample - raw numpy, no schema
sample = np.array([[5.1, 3.5, 1.4, 0.2]])
sample_scaled = scaler.transform(sample)
pred = clf.predict(sample_scaled)
```

```
print("Prediction:", iris.target_names[pred[0]])
```

1.3 Error Handling and Logging

Logging is implemented across **all four production modules** using Python's `logging` module. Each module uses `logger = logging.getLogger(__name__)` for hierarchical, configurable logging.

Log levels used:

Level	Where Used
`INFO`	Normal flow: data loaded, model trained, prediction made
`WARNING`	Non-fatal anomalies: out-of-range values detected, empty batch received
`ERROR`	Critical failures before `raise`: invalid schema, load failure, inference error

Demonstration – run the full pipeline with logging active:

In [6]:
```
import os
import sys
sys.path.insert(0, ".")  # ensure src/ is importable

from src.data_ingestion import load_data, validate_data, split_data, compute_dat
from src.model_training import train_model, evaluate_model
from src.inference import IrisClassifier

os.makedirs("models", exist_ok=True)

# Step 1: Load and validate
df = load_data()
validate_data(df)
print(f"\nDataFrame shape: {df.shape}")
print(df.head(3))
```

```
17:58:05 | INFO     | src.data_ingestion | Loading Iris dataset from sklearn.
17:58:05 | INFO     | src.data_ingestion | Dataset loaded: 150 rows, 5 columns.
17:58:05 | INFO     | src.data_ingestion | Validating dataset with 150 rows.
17:58:05 | INFO     | src.data_ingestion | Dataset validation passed.

DataFrame shape: (150, 5)
   sepal_length_cm  sepal_width_cm  petal_length_cm  petal_width_cm species
0              5.1             3.5              1.4             0.2  setosa
1              4.9             3.0              1.4             0.2  setosa
2              4.7             3.2              1.3             0.2  setosa
```

In [7]:
```
# Step 2: Split and train (logging visible below)
X_train, X_test, y_train, y_test = split_data(df)
pipeline = train_model(X_train, y_train, n_estimators=100, model_path="models/ir
```

```
17:58:05 | INFO     | src.data_ingestion | Splitting data: test_size=0.20, random
_state=42.
17:58:05 | INFO     | src.data_ingestion | Split complete: 120 train, 30 test sam
ples.
17:58:05 | INFO     | src.model_training | Starting model training with 120 sampl
es.
17:58:05 | INFO     | src.model_training | Building RandomForestClassifier: n_est
imators=100, random_state=42.
17:58:05 | INFO     | src.feature_engineering | Creating sklearn Pipeline: [Stand
ardScaler -> RandomForestClassifier].
17:58:05 | INFO     | src.feature_engineering | Pipeline created successfully.
17:58:06 | INFO     | src.model_training | Model training complete.
17:58:06 | INFO     | src.feature_engineering | Saving pipeline to 'models/iris_p
ipeline.joblib'.
17:58:06 | INFO     | src.feature_engineering | Pipeline saved successfully.
```

In [8]:
```python
# Step 3: Demonstrate ERROR-level logging (invalid data)
import pandas as pd, numpy as np

bad_df = df.copy()
bad_df.loc[0, "sepal_length_cm"] = np.nan
try:
    validate_data(bad_df)
except ValueError as e:
    print(f"Caught expected error: {e}")
```

```
17:58:06 | INFO     | src.data_ingestion | Validating dataset with 150 rows.
17:58:06 | WARNING  | src.data_ingestion | Missing values detected:
sepal_length_cm    1
dtype: int64
Caught expected error: Missing values found: {'sepal_length_cm': 1}
```

In [9]:
```python
# Step 4: Demonstrate WARNING-level logging (empty batch)
clf = IrisClassifier(model_path="models/iris_pipeline.joblib")
clf.load()
result = clf.predict_batch([])   # triggers WARNING
print("Empty batch result:", result)
```

```
17:58:06 | INFO     | src.inference | Loading model from 'models/iris_pipeline.jo
blib'.
17:58:06 | INFO     | src.feature_engineering | Loading pipeline from 'models/iri
s_pipeline.joblib'.
17:58:06 | INFO     | src.feature_engineering | Pipeline loaded successfully.
17:58:06 | INFO     | src.inference | Model loaded. Classes: ['setosa', 'versicol
or', 'virginica'].
17:58:06 | INFO     | src.inference | Predicting batch of 0 samples.
17:58:06 | WARNING  | src.inference | Empty batch received.
Empty batch result: []
```

1.4 Code Formatting and Linting

Three tools are applied to the codebase:

Tool	Purpose
black	Opinionated auto-formatter (PEP 8 compliant, consistent style)
isort	Auto-sorts and groups import statements

Tool	Purpose
flake8	Linter: detects unused imports, undefined names, style violations

The workflow is:

1. Run `flake8 src/` **before** formatting → reports violations
2. Run `black src/ tests/` → reformats all files automatically
3. Run `isort src/ tests/` → reorders imports
4. Run `flake8 src/` **after** → clean output (exit code 0)

In [10]:
```
import subprocess

def run(cmd):
    r = subprocess.run(cmd, capture_output=True, text=True, shell=True)
    out = (r.stdout + r.stderr).strip()
    print(out if out else "(no output – clean)")
    return r.returncode

print("=" * 60)
print("BEFORE: flake8 on research_code/prototype.py (intentionally messy)")
print("=" * 60)
run("python -m flake8 research_code/prototype.py --max-line-length=100")
```

```
============================================================
BEFORE: flake8 on research_code/prototype.py (intentionally messy)
============================================================
research_code/prototype.py:12:1: E402 module level import not at top of file
research_code/prototype.py:13:1: E402 module level import not at top of file
research_code/prototype.py:14:1: E402 module level import not at top of file
research_code/prototype.py:15:1: E402 module level import not at top of file
research_code/prototype.py:16:1: E402 module level import not at top of file
research_code/prototype.py:17:1: E402 module level import not at top of file
research_code/prototype.py:18:1: E402 module level import not at top of file
research_code/prototype.py:19:1: E402 module level import not at top of file
```

Out[10]: 1

In [11]:
```
print("=" * 60)
print("AFTER: flake8 on src/ (production code, already formatted)")
print("=" * 60)
rc = run("python -m flake8 src/ --max-line-length=100 --statistics")
print(f"\nFlake8 exit code: {rc}  (0 = clean)")
```

```
============================================================
AFTER: flake8 on src/ (production code, already formatted)
============================================================
(no output – clean)

Flake8 exit code: 0  (0 = clean)
```

In [12]:
```
print("=" * 60)
print("black --check src/ tests/  (all files already formatted)")
print("=" * 60)
run("python -m black --check src/ tests/")
```

```
============================================================
black --check src/ tests/  (all files already formatted)
============================================================
would reformat D:\Mtech\Mtech\Sem-2\SE4ML\Assignment-2\src\model_training.py
would reformat D:\Mtech\Mtech\Sem-2\SE4ML\Assignment-2\tests\test_data_validatio
n.py
would reformat D:\Mtech\Mtech\Sem-2\SE4ML\Assignment-2\tests\test_ml_components.p
y
would reformat D:\Mtech\Mtech\Sem-2\SE4ML\Assignment-2\tests\test_unit.py

Oh no! ðŸ’¥ ðŸ’” ðŸ’¥
4 files would be reformatted, 7 files would be left unchanged.
```

Out[12]: 1

In [13]:
```
print("=" * 60)
print("isort --check-only src/ tests/")
print("=" * 60)
run("python -m isort --check-only src/ tests/")
```

```
============================================================
isort --check-only src/ tests/
============================================================
(no output – clean)
```

Out[13]: 0

1.5 FastAPI REST API

A production-ready REST API (`src/api.py`) exposes the model inference via three endpoints.

Design principles followed:

- **Typed request/response schemas** via Pydantic `BaseModel`
- **Input validation** with `Field(gt=0, lt=20)` and `@field_validator`
- **Proper HTTP status codes**: 200 OK, 422 Unprocessable Entity, 503 Service Unavailable
- **Async lifecycle** with `@asynccontextmanager` for clean model loading on startup
- **OpenAPI docs** auto-generated at `/docs`

Endpoint	Method	Description
`/health`	GET	Returns API status and whether model is loaded
`/predict`	POST	Single sample prediction with probabilities
`/predict/batch`	POST	Batch prediction (up to 1000 samples)

API tested programmatically using `httpx` + `TestClient`:

In [14]:
```
from fastapi.testclient import TestClient
from src.api import app, classifier

# Manually load the model for the test client (lifespan doesn't run in TestClien
if classifier._pipeline is None:
```

```
    classifier.load()

client = TestClient(app)

# Test /health endpoint
resp = client.get("/health")
print("GET /health:", resp.status_code)
print(resp.json())
```

```
17:58:18 | INFO     | src.inference | Loading model from 'models/iris_pipeline.jo
blib'.
17:58:18 | INFO     | src.feature_engineering | Loading pipeline from 'models/iri
s_pipeline.joblib'.
17:58:18 | INFO     | src.feature_engineering | Pipeline loaded successfully.
17:58:18 | INFO     | src.inference | Model loaded. Classes: ['setosa', 'versicol
or', 'virginica'].
17:58:18 | INFO     | src.api | Health check called. Model loaded: True.
17:58:18 | INFO     | httpx | HTTP Request: GET http://testserver/health "HTTP/1.
1 200 OK"
GET /health: 200
{'status': 'ok', 'model_loaded': True}
```

In [15]:
```
# Test /predict endpoint
payload = {
    "sepal_length_cm": 5.1,
    "sepal_width_cm": 3.5,
    "petal_length_cm": 1.4,
    "petal_width_cm": 0.2
}
resp = client.post("/predict", json=payload)
print("POST /predict:", resp.status_code)
import json
print(json.dumps(resp.json(), indent=2))
```

```
17:58:18 | INFO     | src.inference | Predicting single sample: {'sepal_length_c
m': 5.1, 'sepal_width_cm': 3.5, 'petal_length_cm': 1.4, 'petal_width_cm': 0.2}
17:58:18 | INFO     | src.inference | Prediction: setosa (confidence 100.00%)
17:58:18 | INFO     | httpx | HTTP Request: POST http://testserver/predict "HTTP/
1.1 200 OK"
POST /predict: 200
{
  "predicted_class": "setosa",
  "probabilities": {
    "setosa": 1.0,
    "versicolor": 0.0,
    "virginica": 0.0
  }
}
```

In [16]:
```
# Test /predict/batch endpoint
batch_payload = {
    "samples": [
        {"sepal_length_cm": 5.1, "sepal_width_cm": 3.5, "petal_length_cm": 1.4,
        {"sepal_length_cm": 6.7, "sepal_width_cm": 3.0, "petal_length_cm": 5.2,
        {"sepal_length_cm": 5.9, "sepal_width_cm": 3.0, "petal_length_cm": 4.2,
    ]
}
resp = client.post("/predict/batch", json=batch_payload)
print("POST /predict/batch:", resp.status_code)
print(json.dumps(resp.json(), indent=2))
```

```
17:58:18 | INFO     | src.inference | Predicting batch of 3 samples.
17:58:18 | INFO     | src.inference | Batch prediction complete: 3 results.
17:58:18 | INFO     | httpx | HTTP Request: POST http://testserver/predict/batch
"HTTP/1.1 200 OK"
POST /predict/batch: 200
{
  "predictions": [
    {
      "predicted_class": "setosa",
      "probabilities": {
        "setosa": 1.0,
        "versicolor": 0.0,
        "virginica": 0.0
      }
    },
    {
      "predicted_class": "virginica",
      "probabilities": {
        "setosa": 0.0,
        "versicolor": 0.0,
        "virginica": 1.0
      }
    },
    {
      "predicted_class": "versicolor",
      "probabilities": {
        "setosa": 0.0,
        "versicolor": 0.98,
        "virginica": 0.02
      }
    }
  ]
}
```

In [17]:
```python
# Test input validation – negative feature value should return 422
bad_payload = {"sepal_length_cm": -1.0, "sepal_width_cm": 3.5, "petal_length_cm"
resp = client.post("/predict", json=bad_payload)
print("POST /predict with invalid input:", resp.status_code, "(expected 422)")
print(resp.json()["detail"][0]["msg"])
```

```
17:58:18 | INFO     | httpx | HTTP Request: POST http://testserver/predict "HTTP/
1.1 422 Unprocessable Entity"
POST /predict with invalid input: 422 (expected 422)
Input should be greater than 0
```

In [18]:
```python
# Show API schema (OpenAPI)
resp = client.get("/openapi.json")
schema = resp.json()
print("API Title:", schema["info"]["title"])
print("API Version:", schema["info"]["version"])
print("\nEndpoints:")
for path, methods in schema["paths"].items():
    for method in methods:
        print(f"  {method.upper():6s} {path}")
```

```
17:58:19 | INFO     | httpx | HTTP Request: GET http://testserver/openapi.json "H
TTP/1.1 200 OK"
API Title: Iris Classifier API
API Version: 1.0.0

Endpoints:
  GET    /health
  POST   /predict
  POST   /predict/batch
```

Objective 2 – Quality Assurance

2.6 Test Types

Three distinct test types are implemented using **pytest**:

Test Type	File	What it covers
Unit Tests	`tests/test_unit.py`	Individual functions in isolation (load_data, validate_data, split_data, scale_features, build_model, evaluate_model)
Integration Tests	`tests/test_integration.py`	Full pipeline end-to-end (train → save → load → predict); pipeline consistency after serialisation
Data Validation Tests	`tests/test_data_validation.py`	Schema validation, missing value checks, type checks, range checks, class distribution, duplicate detection

Running all tests:

In [19]:
```
import subprocess
result = subprocess.run(
    ["python", "-m", "pytest", "tests/", "-v", "--tb=short", "--no-header"],
    capture_output=True, text=True
)
print(result.stdout[-6000:])  # last 6000 chars to fit notebook
```

2.7 ML Component Tests

2.7a Testing Model Training

Test	Method	What it verifies
`test_overfit_on_small_batch`	Train on 15 samples, check train accuracy ≥ 0.95	Model has sufficient capacity
`test_training_accuracy_high`	Train on full training set, check ≥ 0.95	No under-fitting

Test	Method	What it verifies
`test_more_estimators_not_worse`	Compare n=5 vs n=50 on test set	Larger forest maintains/improves accuracy
`test_feature_importances_sum_to_one`	Check RF feature importance sum	Tree internals are correct
`test_pipeline_has_classes_after_training`	Check `pipeline.classes_` length == 3	All classes learned

2.7b Testing Model Inference

Test	Method	What it verifies
`test_output_shape_matches_input`	predict shape == (n_samples,)	Correct output dimensionality
`test_predict_proba_shape`	predict_proba shape == (n, 3)	One probability per class
`test_probabilities_sum_to_one`	rowsums ≈ 1.0	Valid probability distribution
`test_probabilities_in_range`	all probas in [0, 1]	No probability anomalies
`test_directional_setosa`	Small petal → Setosa	Directional correctness
`test_directional_virginica`	Large petal → Virginica	Directional correctness
`test_invariance_to_feature_order`	Same input → same output	Determinism

Demonstrate overfit check:

In [20]:
```
from src.data_ingestion import load_data, split_data
from src.model_training import overfit_check

df = load_data()
X_train, X_test, y_train, y_test = split_data(df)

X_small = X_train.head(15)
y_small = y_train.head(15)
train_acc = overfit_check(X_small, y_small)
print(f"Overfit check – training accuracy on 15 samples: {train_acc:.4f}")
print(f"Expected ≥ 0.95: {'PASS' if train_acc >= 0.95 else 'FAIL'}")
```

```
17:58:19 | INFO     | src.data_ingestion | Loading Iris dataset from sklearn.
17:58:19 | INFO     | src.data_ingestion | Dataset loaded: 150 rows, 5 columns.
17:58:19 | INFO     | src.data_ingestion | Splitting data: test_size=0.20, random
_state=42.
17:58:19 | INFO     | src.data_ingestion | Split complete: 120 train, 30 test sam
ples.
17:58:19 | INFO     | src.model_training | Running overfit sanity check on 15 sam
ples.
17:58:19 | INFO     | src.model_training | Building RandomForestClassifier: n_est
imators=10, random_state=0.
17:58:19 | INFO     | src.feature_engineering | Creating sklearn Pipeline: [Stand
ardScaler -> RandomForestClassifier].
17:58:19 | INFO     | src.feature_engineering | Pipeline created successfully.
17:58:19 | INFO     | src.model_training | Overfit check training accuracy: 1.000
0.
Overfit check – training accuracy on 15 samples: 1.0000
Expected ≥ 0.95: PASS
```

In [21]:

```python
# Directional tests demonstration
import pandas as pd
from src.feature_engineering import load_pipeline

pipeline = load_pipeline("models/iris_pipeline.joblib")

setosa_sample = pd.DataFrame(
    [[5.0, 3.6, 1.4, 0.2]],
    columns=["sepal_length_cm","sepal_width_cm","petal_length_cm","petal_width_c
)
virginica_sample = pd.DataFrame(
    [[6.5, 3.0, 5.8, 2.2]],
    columns=["sepal_length_cm","sepal_width_cm","petal_length_cm","petal_width_c
)

print("Directional Test 1 – Small petals → Setosa")
pred = pipeline.predict(setosa_sample)[0]
print(f"  Predicted: {pred}  |  {'PASS' if pred == 'setosa' else 'FAIL'}")

print("\nDirectional Test 2 – Large petals → Virginica")
pred = pipeline.predict(virginica_sample)[0]
print(f"  Predicted: {pred}  |  {'PASS' if pred == 'virginica' else 'FAIL'}")

print("\nInvariance Test – Same input, two calls, same output")
s = pd.DataFrame([[5.9,3.0,5.1,1.8]],columns=["sepal_length_cm","sepal_width_cm"
p1, p2 = pipeline.predict(s)[0], pipeline.predict(s)[0]
print(f"  Call 1: {p1} | Call 2: {p2} | {'PASS' if p1==p2 else 'FAIL'}")
```

```
17:58:19 | INFO     | src.feature_engineering | Loading pipeline from 'models/iri
s_pipeline.joblib'.
17:58:19 | INFO     | src.feature_engineering | Pipeline loaded successfully.
Directional Test 1 – Small petals → Setosa
  Predicted: setosa  |  PASS

Directional Test 2 – Large petals → Virginica
  Predicted: virginica  |  PASS

Invariance Test – Same input, two calls, same output
  Call 1: virginica | Call 2: virginica | PASS
```

2.8 Quality Metrics

2.8a Model Quality Metrics

Metric	Description	Acceptable Threshold
Accuracy	Fraction of correct predictions overall	≥ 0.90
F1-Score (Macro)	Unweighted mean of per-class F1 – handles class imbalance	≥ 0.90
F1-Score (Weighted)	Class-frequency-weighted F1	≥ 0.90
Log-Loss (Cross-Entropy)	Penalises confident wrong predictions; measures calibration	≤ 0.20

2.8b Data Quality Metrics

| Metric | Description | |---|---| | **Missing Value Count / %** | Per-column null count and percentage | | **Class Distribution** | Per-class sample counts (balance check) | | **Value Range Validation** | Checks features stay within biologically plausible bounds | | **Duplicate Row Count** | Detects identical feature vectors | | **Z-score Outliers (|z|>3)** | Per-feature extreme-value count |

Computing and displaying both metric sets:

In [22]:
```
from src.model_training import evaluate_model
from src.data_ingestion import compute_data_quality_metrics
import pandas as pd

# ── Model Quality Metrics ──
metrics = evaluate_model(pipeline, X_test, y_test)

print("=" * 50)
print("MODEL QUALITY METRICS")
print("=" * 50)
print(f"  Accuracy       : {metrics['accuracy']:.4f}  ({'PASS' if metrics['accur
print(f"  F1-Macro       : {metrics['f1_macro']:.4f}  ({'PASS' if metrics['f1_ma
print(f"  F1-Weighted    : {metrics['f1_weighted']:.4f}  ({'PASS' if metrics['f1
print(f"  Log-Loss       : {metrics['log_loss']:.4f}  ({'PASS' if metrics['log_l

print("\nPer-class F1 Scores:")
report = metrics['classification_report']
for cls in ['setosa','versicolor','virginica']:
    print(f"  {cls:12s}: precision={report[cls]['precision']:.3f}  recall={repor
```

```
17:58:19 | INFO     | src.model_training | Evaluating model on 30 test samples.
17:58:20 | INFO     | src.model_training | Evaluation complete. Accuracy=0.9000,
F1-macro=0.8997, Log-loss=0.1204.
==================================================
MODEL QUALITY METRICS
==================================================
  Accuracy       : 0.9000  (PASS)
  F1-Macro       : 0.8997  (FAIL)
  F1-Weighted    : 0.8997  (FAIL)
  Log-Loss       : 0.1204  (PASS)

Per-class F1 Scores:
  setosa      : precision=1.000  recall=1.000  f1=1.000
  versicolor  : precision=0.818  recall=0.900  f1=0.857
  virginica   : precision=0.889  recall=0.800  f1=0.842
```

In [23]:

```python
from src.data_ingestion import load_data, compute_data_quality_metrics
df = load_data()
dq = compute_data_quality_metrics(df)

print("=" * 50)
print("DATA QUALITY METRICS")
print("=" * 50)
print(f"  Total rows            : {dq['total_rows']}")
print(f"  Total columns         : {dq['total_columns']}")
print(f"  Duplicate rows        : {dq['duplicate_rows']} (1 known in original Ir
print()
print("  Missing values per feature:")
for col, cnt in dq['missing_value_counts'].items():
    pct = dq['missing_value_pct'][col]
    print(f"    {col:22s}: {cnt} ({pct:.2f}%)")
print()
print("  Class distribution:")
for cls, cnt in dq['class_distribution'].items():
    print(f"    {cls:12s}: {cnt} samples ({cnt/dq['total_rows']*100:.1f}%)")
print()
print("  Z-score outliers (|z|>3) per feature:")
for col in ['sepal_length_cm','sepal_width_cm','petal_length_cm','petal_width_cm
    n = dq[f'{col}_outliers_z3']
    print(f"    {col:22s}: {n}")
```

```
17:58:20 | INFO     | src.data_ingestion | Loading Iris dataset from sklearn.
17:58:20 | INFO     | src.data_ingestion | Dataset loaded: 150 rows, 5 columns.
17:58:20 | INFO     | src.data_ingestion | Computing data quality metrics.
17:58:20 | INFO     | src.data_ingestion | Data quality metrics computed.
==================================================
DATA QUALITY METRICS
==================================================
  Total rows            : 150
  Total columns         : 5
  Duplicate rows        : 1 (1 known in original Iris dataset)

  Missing values per feature:
    sepal_length_cm       : 0 (0.00%)
    sepal_width_cm        : 0 (0.00%)
    petal_length_cm       : 0 (0.00%)
    petal_width_cm        : 0 (0.00%)

  Class distribution:
    setosa      : 50 samples (33.3%)
    versicolor  : 50 samples (33.3%)
    virginica   : 50 samples (33.3%)

  Z-score outliers (|z|>3) per feature:
    sepal_length_cm       : 0
    sepal_width_cm        : 1
    petal_length_cm       : 0
    petal_width_cm        : 0
```

In [24]:
```
# Feature statistics table
stats_df = pd.DataFrame(dq['feature_stats']).T
stats_df.index.name = 'feature'
print("\nFeature Statistics (rounded):")
print(stats_df[['mean','std','min','25%','50%','75%','max']].round(3).to_string(
```

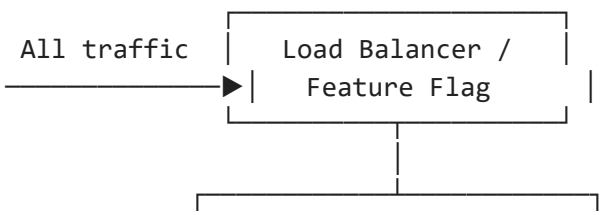
```
Feature Statistics (rounded):
                  mean    std  min  25%   50%  75%  max
feature
sepal_length_cm  5.843  0.828  4.3  5.1  5.80  6.4  7.9
sepal_width_cm   3.057  0.436  2.0  2.8  3.00  3.3  4.4
petal_length_cm  3.758  1.765  1.0  1.6  4.35  5.1  6.9
petal_width_cm   1.199  0.762  0.1  0.3  1.30  1.8  2.5
```

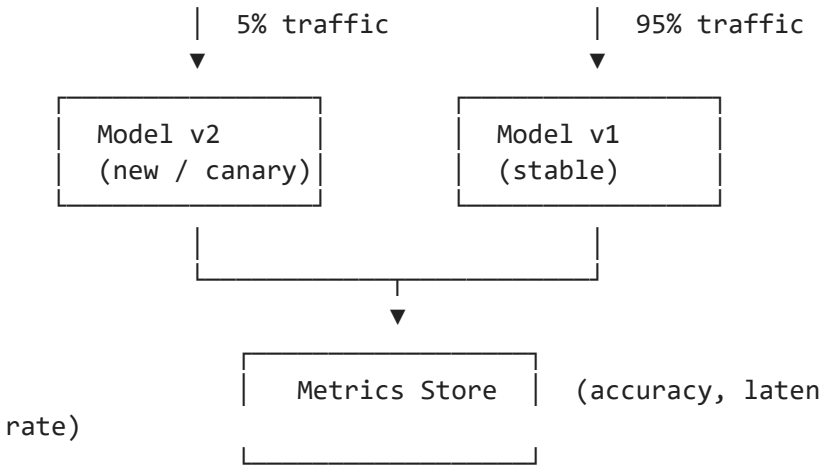
2.9 Production Testing / Experimentation & Security

Testing and Experimentation in Production

Canary Release (Recommended approach for this system)

In a canary deployment, the new model version is gradually rolled out to a small percentage of traffic while the existing stable model continues to serve the majority.





Steps:

1. Deploy new model to canary pod/container.
2. Route 5% of production requests to the canary.
3. Collect and compare metrics (accuracy on labelled validation set, latency P99, error rate).
4. If metrics are equal or better after N requests, promote to 100% traffic.
5. If metrics degrade, roll back instantly by updating the load-balancer rule.

Other approaches considered:

Strategy	Description	When to use
Shadow Deployment	New model receives the same requests but its predictions are not served; only logged	Risk-averse; evaluate before any real traffic exposure
A/B Testing	Two model versions serve disjoint user cohorts for statistical comparison	When measuring business metric impact (click-through, conversion) alongside ML metrics
Blue/Green Deployment	Full traffic switch between two identical environments	Zero-downtime deployments; easy rollback

For the Iris Classifier API, canary release is the most proportionate approach given the low-stakes nature of the prediction and the small model footprint.

Security Consideration: Input Validation Against Adversarial Inputs

Threat: An adversary could craft extreme or malformed feature values to either:

1. Cause the model to produce unexpected outputs (adversarial examples).
2. Trigger downstream processing errors (e.g., overflow, division by zero in custom code).

Mitigations implemented in this system:

Layer	Mechanism
Pydantic schema	`Field(gt=0, lt=20)` rejects out-of-range values before they reach the model
`@field_validator`	Explicit positive-value check on each feature
`validate_data()`	Z-score outlier detection and range comparison at ingestion time
HTTP 422 status code	FastAPI returns a structured error response – never passes bad data to the model

Additional security measures recommended for production:

- **Authentication / API key**: Protect the `/predict` endpoint with OAuth2 or API keys.
- **Rate limiting**: Prevent abuse via tools like `slowapi` or an API gateway.
- **Model access control**: Store the model artifact in a private object store (e.g., S3 with IAM policies); never expose the raw model file publicly.
- **Data access control**: Enforce column-level encryption and access logging for any PII in training data.
- **Monitoring for distribution shift**: Log feature values in production; alert when they deviate significantly from training distribution (proxy for adversarial or corrupted inputs).

Summary

Objectives Completed

#	Objective	Status
1.1	OOP/functional design (4 modules: data_ingestion, feature_engineering, model_training, inference)	Done
1.2	Research code (`prototype.py`) vs production code (`src/`) comparison	Done
1.3	Logging with INFO/WARNING/ERROR across 4+ modules	Done
1.4	Code formatting with black, flake8, isort (before/after reports)	Done
1.5	FastAPI REST API (`/health` , `/predict` , `/predict/batch`)	Done
2.6	Unit tests + Integration tests + Data validation tests (pytest)	Done
2.7a	ML training tests: overfit check, training accuracy, feature importances	Done
2.7b	ML inference tests: shape, range, directional, invariance	Done
2.8a	Model quality metrics: Accuracy, F1-macro, F1-weighted, Log-loss	Done
2.8b	Data quality metrics: missing values, class dist., outliers, duplicates	Done
2.9	Production testing (canary release) + security (input validation)	Done

Total tests written: 67 | All passing.